Engineering-Konsole: Betrieb, Freigabe und Analyse

Version Crime_Data_2026-10-26_v1.0 · Stand 26. Oktober 2026

Diese Anleitung beschreibt die integrierte Konsole auf Basis des bereitgestellten Infrastrukturstands und der Konsole v3.0. Sie ersetzt die Start- und Integrationsanweisungen der alten Konsolenlieferung. Die Infrastruktur bleibt Grundlage: SeaweedFS/S3, bestehende Pipelines, Elasticsearch, Kibana, Jupyter und das vorhandene Monitoring.

1. Was sich durch die Integration ändert

Die Konsole steuert Änderungen und Freigaben. Ein einzelner Worker führt Betriebsimporte, isolierte Kandidatenbauten und die Analysebereitstellung nacheinander aus. Webserver und Worker sind getrennte Prozesse. Die Weboberfläche führt keinen ETL-Lauf selbst aus.

Ein aktiver Datenrelease ist ein unveränderliches Verzeichnis mit Manifest und SHA-256-Prüfsummen. Seine Kennung ist ein Name, nicht selbst die Prüfsumme. AKTUELL.json verweist auf den aktivierten Stand. Der bisherige Stand bleibt aktiv, solange ein neuer Kandidat nur berechnet, geprüft oder auf einen Merge wartet.

Die Konsole unterscheidet jetzt ausdrücklich:

- **Aktiver Release:** freigegebener Datenstand; bei der Erstübernahme gegebenenfalls dokumentierte Alt-Baseline.

- **Analyse-Bereitstellung:** Zustand des Exports dieses Standes nach Elasticsearch/Kibana und der abschliessenden Prüfung.

- **Zuletzt vollständig geprüft:** letzter Stand, dessen gesamte Analysebereitstellung erfolgreich beendet wurde.

Zwischen Datenfreigabe und fertig aktualisierten Analysediensten kann Zeit liegen. Elasticsearch wechselt seine Tabellen-Aliase einzeln. Ein Abbruch kann deshalb einen Teilstand hinterlassen; PARTIAL kennzeichnet ihn. Es gibt keine verteilte Transaktion über alle Dienste.

2. Die Funktionen und ihr Zweck

Aktiver Release und Fortschrittsanzeige

Die Anzeige beantwortet: Auf welchen Stand darf sich eine Analyse beziehen, und wo steht ein Auftrag? Ein Rebuild durchläuft Bronze-Snapshot, Silver einschliesslich Quellhistorisierung, Gold, Serving, Quality Gate, Impact und Freigabe/Release. Angezeigt werden nachgewiesene Schritte, keine erfundene Prozentzahl oder Restlaufzeit. Mehrere Vintages werden innerhalb derselben Stufe bearbeitet. SUCCEEDED eines Imports bedeutet nur, dass die Quelle geladen wurde.

Beispiel: Detroit Full ist erfolgreich, der Rebuild wartet auf Review. Bronze ist geladen; der aktive Release bleibt der vorherige. Eine grüne Importzeile ist keine fachliche Freigabe.

Mapping vorschlagen

Ein Engineer wählt Stadt, Quellcode und NIBRS-Zielcode, nennt eine Begründung und reicht den Vorschlag ein. Die Konsole speichert eine neue Revision mit Bezug auf die aktuelle Basis und berechnet einen Kandidaten. Führende Nullen in Quellcodes bleiben erhalten. Ein fachlich entschiedenes Mapping erhält admin_review und wird als ambiguous eingeordnet: Die Entscheidung ist geprüft, aber kein amtlicher Zuordnungsnachweis.

Konfiguration und Zeitpläne

Die JSON-Konfiguration ist die gemeinsame Quelle für Zeitraum, Qualitätsgrenze und Jobs. Die Quellimport-Formulare bearbeiten dieselbe Konfiguration. Sie stellen keine zweite unabhängige Konfiguration dar. Änderungen durchlaufen ebenfalls Kandidat und Review.

scope_start ist inklusive, scope_end_exclusive exklusiv. Der Zeitraum bestimmt die Jahre des verwalteten Gesamtimports und die Census-Vintages des Rebuilds. Er filtert nicht nachträglich beliebige bereits geladene Bronze-Jahre weg: Daten ausserhalb des festgelegten Zeitraums werden im Quality Gate beanstandet.

Auf srv01 bleibt der bestehende Wochenrhythmus über einen angepassten Cron-Einstieg zuständig. ADMIN_SCHEDULER=false deaktiviert den internen Konsolen-Scheduler. Dadurch sind Zeitplanfelder in der Konsole zwar gespeichert, werden dort aber nicht ausgeführt. Für einen bewusst ausschliesslich konsolengesteuerten Betrieb kann der interne Scheduler aktiviert werden; dann muss der externe Cron-Einstieg deaktiviert sein. Es darf genau einen Zeitplanverantwortlichen geben.

Quellimporte und Ladelogik

Chicago Full lädt einen neuen Vollsnapshot für ein Tatjahr. Chicago Delta verwendet updated_on, das Wasserzeichen und den im Pipeline-Code festgelegten Lookback von 14 Tagen. Ohne Wasserzeichen entsteht zunächst ein Vollsnapshot. Der aktuelle Chicago-CLI besitzt keinen frei übergebbaren --lookback-days-Schalter.

Detroit Full lädt die angegebenen Jahre vollständig. Detroit Delta arbeitet je Jahr mit updated_at, Wasserzeichen und Pipeline-Lookback. Der ältere Daily-Modus verwendet dagegen ein Kalenderfenster bis zum abgeschlossenen Vortag. Er bleibt zur Kompatibilität vorhanden und ist als Legacy bezeichnet. Für den regulären Betrieb ist Delta vorgesehen.

Die Stadtbuttons öffnen die lesende Codeansicht. Full und Delta markieren relevante unterschiedliche Codebereiche. Gemeinsame Funktionen werden weiterhin vollständig angezeigt. Suche und Dateiauswahl ändern keinen Code. Die Vorschau zeigt den Code des Webserver-Abbilds. Historischer Worker-Code wird ohne Nachweis nicht behauptet.

Quelllauf-Nachweise enthalten Manifest, Run-ID, Parameter und Zeitstempel, soweit sie im tatsächlichen Manifest stehen. Job- und Impact-Verknüpfungen beruhen auf archivierten Manifest-Hashes. Ein fehlender Verweis bedeutet fehlenden Nachweis. Auch remote gespeicherte Bronze-Manifeste werden in der Konsole gelesen.

Review, Anträge und Konflikte

Die Review-Ansichten zeigen eigene Anträge, offene Reviews und Zuweisungen. Ein Reviewer kann einen Antrag übernehmen, lesen und fachlich entscheiden. Die Funktion ist eine interne Arbeitsliste, kein E-Mail-Postfach. Sie verschickt keine automatische E-Mail. Der Impact steht in der Konsole und als herunterladbares Archiv zur Verfügung.

Konkurrierende Vorschläge auf derselben Mappingzeile oder demselben Konfigurationsfeld werden angezeigt. Veraltete Vorschläge müssen auf die neue Basis übertragen und neu berechnet werden. Die Vier-Augen-Regel verbietet die Freigabe durch den Antragsteller. Eine Freigabe bindet den konkreten Impact-Lauf und seinen Berichtshash.

Jobs und Prüfprotokolle

Quellimporte laden Raw/Bronze. rebuild erstellt einen isolierten Vorschlag. replay verlangt denselben logischen Inhalt wie der aktive Release. quality prüft den verfügbaren Datenstand. Logs enthalten den Pipeline-Schritt und bei erkannten Fehlern die fehlende Abhängigkeit und Abhilfe. Ein Job im Status QUEUED benötigt einen laufenden Worker mit demselben absoluten Datenverzeichnis.

Geplante Gesamtimporte erscheinen unter Betriebsaufträge als batch. Ihr Log liegt unter data/admin/jobs/<Auftrags-ID>/run.log. Nach erfolgreichen Quellenimporten wird automatisch ein Rebuild eingereiht. Der Batch selbst veröffentlicht keinen Datenrelease.

Git und Betrieb

Im gekoppelten Modus gilt: Impact archivieren, fachlich freigeben, PR/CI prüfen, exakten Merge nachweisen, Datenrelease aktivieren. Mapping- und Konfigurationsänderungen benötigen diesen Git-Schritt. Reine neue Quelldaten benötigen unter unveränderten Regeln keinen zusätzlichen PR, aber weiterhin einen geprüften Datenkandidaten. Auch dessen Build-Code wird im gekoppelten Betrieb gegen Main geprüft.

Die Betriebsaktionen umfassen PR-Erstellung, Merge-Anforderung, Rücknahme, erneute Archivierung, Backup und Analysebereitstellung. Der Worker führt sie aus. Repository-Zugriff und Branch-Schutz müssen eingerichtet sein; die Konsole umgeht keine fehlgeschlagenen CI-Prüfungen.

3. Was der Impact-Bericht tatsächlich misst

Der Bericht vergleicht den aktiven Release mit dem Kandidaten: hinzugefügte, entfernte und geänderte Datensätze, Delikte und Fälle, Wechsel von NIBRS-Code und Kategorie sowie Qualitätskennzahlen. Die Zählweise bleibt diejenige der Pipeline: Delikte entsprechen zählenden Gold-Zeilen; Fälle entsprechen counts_case-Repräsentanten. Fallzahlen sind nicht pauschal die Summe beliebiger Kategorien.

Vergleiche sind nach Stadt, Zeitraum und Deliktkategorie aufgeschlüsselt. Die vollständigen Detaildaten liegen als Parquet vor; die Anzeige im Browser ist begrenzt. Zusätzlich werden logische Fingerprints der Gold-/Serving-Tabellen verglichen. Damit werden auch Dimensions- und Nenneränderungen sichtbar. Nicht jedes geänderte Feld erhält automatisch eine eigene fachliche Visualisierung.

Beispiel mit **ausschliesslich fiktiven Zahlen**, keine Mapping-Empfehlung: Ein Detroit-Mapping verschiebt 120 zählende Zeilen aus einer Auffangkategorie in Betrug.

Messgrösse	Vorher	Nachher	Änderung
Detroit, Delikte gesamt	10 000	10 000	0
Auffangkategorie	500	380	-120
Betrug	800	920	+120
Unveränderte Datensätze	9 880	9 880	0
Datensätze mit geändertem Mapping	0	120	+120

Es entsteht kein zusätzliches Delikt. Die Verteilung ändert sich. Auswirkungen auf Fälle, Zeitaggregate, Raten und Qualitätsstatus werden aus den wirklichen Daten berechnet; sie dürfen nicht aus der Beispielverschiebung geraten werden. Ein Erstaufbau ohne Baseline und ein alter Release ohne Silver-Nachweise werden als eingeschränkt vergleichbar gekennzeichnet.

Pro Änderungsantrag und Berechnungsversuch bleiben Bericht, Detailartefakte, Hashes und Freigaben archiviert. Ein fehlgeschlagenes Archivieren verhindert die Freigabe; nach Behebung kann archive-retry verwendet werden.

4. Lokale Tests unter Windows: chronologische Einrichtung

Voraussetzung: Python 3.12 oder 3.13, entpacktes Projekt und freie Platte für Quellen, Snapshot, Kandidat und Release. Für vollständige Quellenimporte werden Netzwerkzugriff und gegebenenfalls API-Schlüssel benötigt. Die Server-Lockdatei requirements.lock ist auf Python 3.13 ausgelegt. Für einen lokalen Einstieg werden die Paketbereiche installiert.

Schritt 1: Projekt und Umgebung vorbereiten

Im Projektordner, in PowerShell:

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -e ".[admin,dev]"
$ProjectRoot = (Get-Location).Path
$DataRoot = Join-Path $ProjectRoot "data"
$env:ADMIN_RELEASE_MODE = "separate"
$env:ADMIN_OBJECT_STORE = "local"
$env:STORAGE_BACKEND = "local"
$env:PYTHON_DOTENV_DISABLED = "1"
$env:DUCKDB_MEMORY_LIMIT = "896MB"
$env:DUCKDB_THREADS = "2"
$env:DUCKDB_TEMP_DIRECTORY = Join-Path $DataRoot "duckdb-tmp"
```

Korrekt ist -e als zusammenhängende Option; nicht - e. Die direkte Ausführung des venv-Python benötigt keine Activate.ps1 und keine Änderung der PowerShell-Ausführungsrichtlinie. Die Speichergrenze ist eine Startkonfiguration, keine Zusage, dass jeder Full Load damit erfolgreich ist.

Schritt 2: Zeitraum festlegen, dann initialisieren

Für die Mehrjahresstrecke 2021 bis 2026 kann das Beispiel verwendet werden:

```
Copy-Item .\docs\console-settings.example.json `
  .\config\platform_admin_settings.json
.\.venv\Scripts\python.exe -m platform_admin init `
  --project-root "$ProjectRoot" --data-root "$DataRoot" `
  --operator Bootstrap
.\.venv\Scripts\python.exe -m platform_admin enable-managed `
  --project-root "$ProjectRoot" --data-root "$DataRoot" `
  --operator Bootstrap
```

Nur vor der **ersten** Initialisierung kopieren und den Zeitraum fachlich prüfen. init bewahrt vorhandene Revisionen. Bei einer bestehenden Konsole muss eine Konfigurationsänderung regulär vorgeschlagen und freigegeben werden. Ohne Bootstrap-Datei verwendet init den bisherigen lokalen Testzeitraum 2024. enable-managed dokumentiert die Übernahme und sperrt direkte schreibende Pipeline-Einstiege in diesem Datenverzeichnis. Ein vorhandener Release erhält dadurch keine erfundene rückwirkende Freigabe.

Schritt 3: Webserver starten, Terminal offen lassen

Im selben Terminal und mit den oben gesetzten Variablen:

```
.\.venv\Scripts\python.exe -m platform_admin serve `
  --project-root "$ProjectRoot" --data-root "$DataRoot" `
  --operator Engineer --port 8765
```

Die ausgegebene vollständige URL inklusive Sitzungstoken öffnen. Die Engineering-Konsole ist bereits die Flask-Webanwendung; es gibt lokal keinen dritten separat zu startenden Flask-Webserver. Dieser lokale Server bindet nur 127.0.0.1.

Schritt 4: Worker in einem zweiten Terminal starten

Im selben Projektordner zuerst die Variablen aus Schritt 1 erneut setzen; Umgebungsvariablen gelten pro Terminal. Dann:

```
.\.venv\Scripts\python.exe -m platform_admin worker `
  --project-root "$ProjectRoot" --data-root "$DataRoot" `
  --operator Worker
```

Terminal geöffnet lassen. Der Worker verarbeitet die Warteschlange. --once verarbeitet nur einen Tick und ist kein dauerhaft laufender Worker.

Schritt 5: Quellen laden und Rebuild prüfen

Ein Gesamtimport lädt auch die notwendigen Census- und Wetterquellen. In einem weiteren Terminal mit denselben Variablen:

```
.\.venv\Scripts\python.exe -m platform_admin.batch `
  --project-root "$ProjectRoot" --data-root "$DataRoot" `
  --mode full --operator Engineer --slot local-initial-001
```

Der Batch verwendet die genehmigte Konfiguration. Ein erneuter Aufruf mit demselben slot erzeugt keinen Doppelauftrag, auch nach einem Fehler. Für einen bewussten neuen Versuch einen neuen slot verwenden. Einzelne Chicago-/Detroit-Buttons laden nur die jeweilige Kriminalquelle. Für einen vollständigen Rebuild reichen diese beiden Quellen allein nicht aus.

Schritt 6: Zweite Review-Sitzung

```
.\.venv\Scripts\python.exe -m platform_admin serve `
  --project-root "$ProjectRoot" --data-root "$DataRoot" `
  --operator Reviewer --port 8766
```

Die zweite URL öffnen, Antrag und Impact prüfen und freigeben. Der lokale Operatorname ist eine Testidentität, kein authentifizierter Benutzer. Im Serverbetrieb ersetzt Cloudflare Access diese Namensangabe.

Für Tests ohne Elasticsearch/Kibana bleibt ADMIN_AUTO_SERVING ungesetzt. Der freigegebene Release ist dann verfügbar; ein erfolgreicher extern bereitgestellter Analysebestand wird nicht behauptet. Sind die Analysedienste gestartet, ihre regulären ELASTICSEARCH_URL und KIBANA_URL setzen und die Aktion „Such- und Analysebestand aufbauen“ einreihen.

5. Übernahme auf srv01

Die Integration ist für **einen schreibenden Host mit gemeinsamem lokalen Volume** ausgelegt. SQLite gehört auf dieses Volume, nicht auf S3/FUSE. Es wird kein zusätzlicher paralleler Silver-Container gestartet. Webserver und Worker verwenden denselben Code- und Datenstand.

Vorbereitung ohne Änderung des laufenden Betriebs

1. Änderungen aus diesem Paket in das echte Repository übernehmen und prüfen. Den gewünschten Mehrjahreszeitraum in config/platform_admin_settings.json versionieren. Für coupled muss Main dem geprüften Quellstand entsprechen; ein unveröffentlichtes ZIP genügt für diesen Nachweis nicht.

2. Bestehende Kontrolle, Daten und Secrets sichern. Laufende Full Loads, Rebuilds und Veröffentlichungen beenden lassen.

3. Basisabbild **aus diesem integrierten Checkout** bauen. Danach Dockerfile.admin mit genau diesem Basisabbild bauen. Anschliessend Registry-Veröffentlichung und Digest-Pinning nach eurem bestehenden Betriebsverfahren. Auf srv01 keinen ressourcenintensiven Build starten.

```
docker build --build-arg GIT_COMMIT="$(git rev-parse HEAD)" \
  -t urban-crime:integrated .
docker build -f Dockerfile.admin \
  --build-arg PIPELINE_BASE=urban-crime:integrated \
  -t urban-crime-admin:2026-10-26-v1.0 .
```

Die bestehende manuell ausgelöste Image-Pipeline wird nicht in ein automatisches Deployment umgewandelt. Ein Merge allein aktualisiert keinen laufenden Container. Neues Abbild bauen/pushen, Digest anpassen, pull und kontrolliertes Neuerstellen sind weiterhin eigene Betriebsschritte. Nie während eines aktiven Schreibauftrags aktualisieren.

Konfiguration und Berechtigungen

docker-compose.console.yml ergänzt docker-compose.srv01.yml. ADMIN_IMAGE muss den Digest des integrierten Admin-Abbilds enthalten. Geerbte Einstellungen behalten S3, Speichergrenzen, Netz und Volume-Pfade der Zielarchitektur bei. Der Worker erhält keine Docker-Socket-Berechtigung.

Zusätzliche .env-Werte: ADMIN_IMAGE, ADMIN_HOST, ADMIN_PUBLIC_ORIGIN, CF_ACCESS_ISSUER, CF_ACCESS_AUD, ADMIN_ROLE_BINDINGS_PATH, ADMIN_METRICS_TOKEN_PATH, ADMIN_GITHUB_TOKEN_PATH und ADMIN_GITHUB_REPOSITORY. Die bestehenden S3-, Quellen- und Analysedienstvariablen bleiben notwendig. S3_ENDPOINT kann mit oder ohne Protokoll angegeben werden; S3_USE_SSL muss zur Infrastruktur passen.

Die drei *_PATH-Werte zeigen auf lokale Secret-Dateien. roles.example.json zeigt das Format. Rollen werden stabilen Access-sub-Identitäten zugeordnet, nicht frei übergebenen E-Mail-Adressen. Engineer, Reviewer und Release-Manager nach eurem Berechtigungskonzept besetzen. Der GitHub-Token wird nur dem Worker bereitgestellt. Branch-Schutz und verpflichtende CI-Prüfungen müssen im Repository tatsächlich bestehen.

Für den neuen Hostnamen ist eine Cloudflare-Tunnel-/Access-Route auf Traefik erforderlich. Es werden keine Host-Ports der Konsole veröffentlicht. Die Compose-Datei allein richtet die externe Access-Anwendung nicht ein.

Kontrollierter Umschaltablauf

Zuerst den bisherigen Cron-/Timer-Einstieg deaktivieren und sicherstellen, dass keine alten Pipeline-Container mehr schreiben. Nicht beide Skripte parallel einplanen. Danach auf srv01:

```
dc() {
  docker compose --env-file .env \
    -f docker-compose.srv01.yml \
    -f docker-compose.console.yml "$@"
}
dc config --quiet
dc pull pipeline admin worker
dc run --rm --no-deps worker python -m platform_admin init \
  --project-root /app --data-root /app/data --operator Bootstrap
dc run --rm --no-deps worker python -m platform_admin enable-managed \
  --project-root /app --data-root /app/data --operator Bootstrap
dc --profile console up -d admin worker
dc ps
dc logs --tail=100 admin worker
```

enable-managed holt im S3-Betrieb den bestehenden aktiven Release lokal und dokumentiert ihn als Alt-Baseline. Das geschieht vor neuen Vorschlägen. Bestehende SQLite-Revisionen bleiben erhalten. Offene alte Jobs müssen vor der Übernahme kontrolliert abgeschlossen werden. Vor dem ersten neuen Rebuild prüfen, dass die Konfiguration alle vorhandenen Jahre umfasst und ihre Mapping-Basis fachlich zum übernommenen Stand passt; ein altes Release beweist diese Übereinstimmung nicht automatisch.

Erst nach erfolgreichem Start den bisherigen Zeitplaneintrag auf `docker/srv01-console-datenlauf.sh auto` umstellen. Den bisherigen Wochenrhythmus beibehalten. Das Skript reiht nur ein; der persistente Worker verarbeitet. Der zweite geplante Montag im Monat ist Full, die anderen bis Tag 28 Delta, ein fünfter Montag wird übersprungen. Ein manueller Aufruf mit full oder delta ist möglich.

Elasticsearch/Kibana müssen für die automatische Analysebereitstellung laufen. ADMIN_AUTO_SERVING=true reiht nach einer Freigabe genau einen Bereitstellungsauftrag je Release automatisch ein. Ein fehlgeschlagener Auftrag wird nicht endlos automatisch wiederholt; nach Behebung erneut über die Konsole starten.

6. Analyse und Monitoring

Die bestehenden Elasticsearch-Sterntabellen, Geo-Indizes, Raten und Kibana-Dashboards werden durch die vorhandenen Lader erstellt. Die alte vereinfachte Konsolen-Indexstruktur wird nicht mehr verwendet. ClickHouse ist in diesem Zielstand keine Pflicht und wird von diesem Adapter nicht geladen.

Notebook 34 und 37 lesen einen beim Öffnen festgehaltenen Releasepfad. Ein neuer Datenrelease ändert eine laufende Notebook-Sitzung nicht. Für den neuen Stand die Initialisierungszelle erneut ausführen. Bestehende bearbeitete Notebook-Kopien im Jupyter-Arbeitsbereich werden vom Containerstart nicht überschrieben; die aktualisierten Vorlagen bewusst in eine neue Arbeitskopie übernehmen. Für die neuen Vorlagen auch das Jupyter-Abbild aus dem integrierten Stand neu bauen.

Grafana bleibt die Betriebsübersicht für Infrastruktur, Speicher und Dienste. deploy/console/prometheus.example.yml ergänzt einen authentifizierten Scrape-Endpunkt. Prometheus benötigt Netzwerkzugriff auf admin:8765 und das Metrics-Secret. deploy/console/grafana.json kann als zusätzliches Dashboard importiert werden; die vorhandenen Dashboards bleiben erhalten.

Die neuen Metriken zeigen Job- und Betriebsauftragszustände, den letzten Dienststatus und dessen Alter. Ein alter Worker-Status kann während eines langen Jobs entstehen, da der Tick erst danach zurückkehrt; allein

das Alter beweist keinen abgestürzten Worker. Die alten pipeline_run-Metriken dokumentieren weiterhin den alten run_all-Einstieg. Sie werden nicht künstlich als aktueller Konsolenlauf ausgegeben. Nach der Umstellung für den verwalteten Betrieb die Konsolenmetriken verwenden.

7. Fehlerdiagnose und Wiederanlauf

QUEUED bleibt stehen

```
Get-CimInstance Win32_Process |
  Where-Object { $_.CommandLine -match "platform_admin (serve|worker)" } |
  Select-Object ProcessId, ParentProcessId, CommandLine |
  Format-List
Get-NetTCPConnection -LocalPort 8765 -State Listen |
  Select-Object LocalAddress, LocalPort, OwningProcess
```

Mindestens ein Worker muss laufen. Webserver und Worker müssen denselben absoluten project-root und data-root nutzen. Zwei Python-Zeilen können unter Windows ein venv-Launcher und dessen Kindprozess sein; sie beweisen nicht automatisch zwei unabhängige Server. ParentProcessId und Portbesitzer prüfen. Nur die identifizierte überzählige Sitzung beenden, vorzugsweise mit Ctrl+C in ihrem Terminal.

RUNNING bleibt nach einem Abbruch stehen

Zuerst Worker und Pipeline-Kindprozesse prüfen und kontrolliert stoppen. Logs lesen. Ein KeyboardInterrupt bedeutet einen abgebrochenen Lauf, nicht einen noch rechnenden Import. Vor einer manuellen Statuskorrektur die Konsole ebenfalls stoppen und data/admin sichern. Bei hartem Prozessende kann worker.lock liegenbleiben; diese Sperre nur nach nachgewiesenem Ende **aller** Schreiber entfernen. Ein Container-PID ist nicht unbedingt derselbe wie der Host-PID.

```
# Nur nach bestaetigtem Stillstand aller Schreiber:
Copy-Item .\data\admin .\admin-recovery-backup -Recurse
# Nur falls die verwaiste Datei noch existiert:
Remove-Item .\data\admin\worker.lock
.\.venv\Scripts\python.exe -m platform_admin recover-job `
  --project-root "$ProjectRoot" --data-root "$DataRoot" `
  --job-id VOLLSTAENDIGE_JOB_ID --operator Operator `
  --reason "Prozesse beendet; Import nach Abbruch kontrolliert abgeschlossen"
```

recover-job schliesst nur RUNNING-Import-/Prüfjobs ab, keine Impacts. Für einen Impact gibt es abandon-impact mit --impact-id und Begründung. Für eine Betriebsaktion recover-operation mit --operation-id. Danach Worker neu starten und einen **neuen** Auftrag einreihen. Es wird kein halbfertiger Lauf als erfolgreich markiert.

Rebuild scheitert an fehlender Abhängigkeit

Beispiel: „Kein vollständiger Bronze-Lauf für tiger_tract, Vintage 2024, Staat 17“. Es fehlt eine Bronze-Quelle für Silver, nicht einfach eine bereits vorhandene Silver-/Gold-Datei. Ein Gesamtimport mit der konfigurierten Jahresspanne lädt Chicago, Detroit, TIGER Tracts, TIGER Places, ACS und Wetter. Fehlende API-Schlüssel zuerst beheben. Anschliessend neuen Rebuild ausführen. Im verwalteten Betrieb Quellen über Batch oder Konsole starten, nicht direkt am Worker vorbei.

Weitere Fälle: fehlende DuckDB-Spatial-/ICU-Erweiterungen, API 401/403, Rate Limit 429, Netzwerkfehler, fehlende Python-Pakete, Dateiberechtigungen und voller Datenträger. Das konkrete Job-Log enthält die erste relevante Ursache. Bei OOM DuckDB-Grenzen, freien Plattenplatz für Auslagerung und parallele Containerlast prüfen. Ein zusätzlicher Silver-Container vergrössert das verfügbare RAM nicht.

ARCHIVE_PENDING, AWAITING_GIT und PARTIAL

ARCHIVE_PENDING: S3-Zugang und Platz prüfen, danach Archivierung wiederholen. AWAITING_GIT: PR, CI, Branch-Schutz und Merge-Nachweis prüfen; nicht den Freigabemodus umschalten, um den konkreten Auftrag freizubekommen. PARTIAL: Bereitstellungslog unter data/admin/serving/<Release-ID>/run.log prüfen, Analysedienste reparieren und Bereitstellung des aktiven Releases erneut einreihen. Der Datenrelease bleibt dabei unverändert.

Aufräumen und Backup

Im verwalteten Modus bleibt der Aufräum-Trockenlauf lesend verfügbar. Destruktives Aufräumen ist zunächst gesperrt, weil die bisherigen Retentionsregeln offene Kandidaten und Prüfarchive nicht vollständig modellieren. Das ist eine bewusst verbleibende Betriebseinschränkung, keine Behauptung einer fertig integrierten Löschautomatik. Ein Full Load hebt andere Pin-/Referenz-/Historisierungsregeln nicht automatisch auf.

Die Backup-Aktion sichert SQLite konsistent sowie Kandidaten, Revisionen, Impact-Archive, Bereitstellungsnachweise und lokale Releases. Im S3-Modus liegt das Archiv unter backups/control/; der Worker sichert täglich nach einem abgeschlossenen Tick. Lokal liegt es unter data/backups/control/. Das ist keine unabhängige Katastrophensicherung: Bucket bzw. Volume müssen weiterhin in eure externe Backup-Policy aufgenommen sein. Der bisherige kuratierte Sicherungssatz allein umfasst die neue Kontrolldatenbank nicht.

restore-backup verlangt ein leeres Ziel. Nach Restore denselben absoluten Datenpfad verwenden, da die Kontrolldatenbank Pfade enthält. Zuerst Dienste stoppen, Integrität prüfen, ausstehende Operationen kontrollieren und erst dann den Worker starten. Kein automatisches Löschen alter Auditdaten ist enthalten.

8. Was geprüft wurde und was vor Ort offen bleibt

Die Lieferung enthält automatisierte Prüfungen mit echten SQLite-/Parquet-Dateien, produktiven Pipeline-Einstiegspunkten auf kleinen Testdaten und kontrollierten S3-/Git-/Serving-Adaptern. Der gesonderte TESTNACHWEIS nennt die tatsächlich ausgeführten Läufe und Grenzen.

Nicht ausgeführt wurden ein Deployment auf srv01, produktive Full Loads, echte Cloudflare-SSO-Anmeldung, GitHub-Merges oder ein Export gegen eure laufenden Elasticsearch-/Kibana-Dienste. Docker/Compose ist in der Prüfungsumgebung nicht vorhanden; das zusammengeführte Compose-Modell und die Container müssen deshalb vor Ort geprüft werden. Ein erfolgreicher Unit-Test ersetzt weder den Lasttest noch den Restore-Test auf eurer Infrastruktur.

9. Auslieferung und nächste Version

Dieses Paket heisst Crime_Data_2026-10-26_v1.0. Eine weitere Lieferung am selben Tag erhält v2.0. Am Folgetag beginnt die Zählung mit dem neuen Datum wieder bei v1.0. Die Paketversion ist unabhängig von Datenrelease-ID, Mappingrevision, Git-Commit und Container-Digest.